

Let's write some pseudocode for how this works.

Listing 9.1. Pseudocode for neighborhood Q-learning, part 1

```
# Initialize actions for all agents

   for j in agents:                                      1
       state = environment.get_state()                   2
       neighbors = get_neighbors(j, num=3)               3
       joint_action = get_joint_action(neighbors)        4
       q_values = Q(state, joint_action)                 5
       j.action = policy(q_values)                       6
       environment.take_action(j.action)
       reward = environment.get_reward()
```

- ***1*** Iterates through all the agents in the environment, stored in a list
- ***2*** Retrieves the current environment state
- ***3*** This function will find the closest 3 agents to agent j.
- ***4*** This function will return the joint action of agent j's neighbors.
- ***5*** Gets the Q values for each action of agent j, given the state and the joint action of its neighbors
- ***6*** This function will return a discrete action using the Q values.

The pseudocode in listing 9.1 shows that we need a function that takes the current agent *j* and finds its nearest three neighbors, and then we need another function that will build the joint action using these three nearest neighbors. At this point, we have another problem: how do we build the joint action without already knowing the actions of the other agents? In order to compute the Q values for agent *j* (and thus take an action), we need to know the actions that agents *–j* are taking (we use *–j* to denote the agents that are *not* agent *j*, but in this case only the nearest neighbors). In order to figure out the actions of agents *–j*, however, we would need to compute all of their Q values, and then it seems like we get into an infinite loop and never get anywhere.